

Reviewer Pack — Lie Nilpotency Class of Layer Algebra Distinguishes Read-Twi...

Entry 11647c329d11 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 08:32:25 UTC

Lie Nilpotency Class of Layer Algebra Distinguishes Read-Twice BPs from IP₂

Entry ID: 11647c329d11 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 08:32:25 UTC

1. Conjecture

Field A (mathematical branch): Engel-Lie nilpotency of finitely-generated matrix Lie subalgebras (descending central series and Lazard-style commutator filtrations from pro-p / Magnus-algebra theory), a classical tool in nilpotent group theory but essentially absent from branching-program lower-bound literature **Field B** (complexity object): Read-twice branching program (BP) size for $IP_{2,n} = \bigoplus_{i=1}^n x_i y_i$, in the Borodin-Razborov-Smolensky 1993 model

Statement:

For a layered BP P with state set S of size s and layers $k=1..L$, let $M_k = M_k^{\{0\}+M_k\{1\}}$ be the (variable-marginalized) integer transition matrix of layer k , set $\bar{M}_k = M_k - (\text{tr } M_k / s) \cdot I_s$, and let $g(P) \subset M_s(Q)$ be the Lie subalgebra generated under $[A,B]=AB-BA$ by $\{\bar{M}_1, \dots, \bar{M}_L\}$. Define the distinguishing tensor $\rho(P) :=$ nilpotency class of $g(P)$ (smallest c such that every $(c+1)$ -fold nested commutator of generators is the zero matrix; $\rho(P)=\infty$ if $g(P)$ is not nilpotent). Conjecture: (A) for every read-twice BP P , $\rho(P) \leq 4 \cdot \log_2(\text{size}(P)) + O(1)$; (B) for the canonical unrestricted product BP for $IP_{2,n}$, $\rho \geq \lceil n/2 \rceil$; together (A)+(B) imply any read-twice BP for $IP_{2,n}$ has size $\geq 2^{\Omega(n)}$.

Rationale (proposer's reasoning):

Reading a variable twice forces the same layer matrix to appear at two filtration depths, which should collapse iterated commutators (cap nilpotency class by log-size), whereas the inner-product BP encodes a non-degenerate symplectic form whose product representation interleaves Heisenberg-type commutators of depth $\Theta(n)$. This is a basis-free combinatorial invariant orthogonal to spectral / free-cumulant approaches that have repeatedly crashed in the blacklist, and it does not rely on any natural-property test on truth-tables, dodging the natural-proofs barrier.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e9139e5ecc5323e8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across $n \in \{3..12\}$ and 30 seeds: (A) $\geq 90\%$ of random read-twice BPs at each $s \in \{4, 8, 16, 32, 64, 128\}$ satisfy $\rho(P) \leq 4 \cdot \log_2(s) + 10$; (B) the canonical IP_2 product BP yields $\rho \geq \lceil n/2 \rceil$ for every tested n . Any violation falsifies.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.78	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - read-twice branching program lower bound inner product IP_2 - Lie nilpotency class transition matrix branching program lower bound - commutator filtration Lazard descending central series circuit complexity

Top relevant hits considered: - [<http://arxiv.org/abs/1709.08890v2>] Partial matching width and its application to lower bounds for branching programs - [<http://arxiv.org/abs/2001.07570v2>] Hom 3-Lie-Rinehart Algebras - [<http://arxiv.org/abs/1807.11011v2>] The capability and certain functors of some nilpotent Lie algebras of class two

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math
from collections import deque

def matrix_add(A, B):
    return [[A[i][j] + B[i][j] for j in range(len(A[0]))] for i in range(len(A))]

def matrix_sub(A, B):
    return [[A[i][j] - B[i][j] for j in range(len(A[0]))] for i in range(len(A))]

def matrix_mul(A, B):
    result = [[sum(A[i][k] * B[k][j] for k in range(len(B))) for j in range(len(B[0]))] for i in range(len(A))]
    return result

def identity_matrix(n):
    return [[1 if i == j else 0 for j in range(n)] for i in range(n)]

def trace(matrix):
    return sum(matrix[i][i] for i in range(len(matrix)))
```

```

def frobenius_norm(matrix):
    return math.sqrt(sum(sum(abs(x) ** 2 for x in row) for row in matrix))

def is_nilpotent(matrix, tolerance=1e-9):
    n = len(matrix)
    identity = identity_matrix(n)
    current = matrix
    for c in range(1, n + 1):
        if frobenius_norm(current) < tolerance:
            return c - 1
        current = matrix_mul(current, matrix_sub(matrix_add(*matrix_bar_k, identity)))
    return float('inf')

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def build_bp(n):
        layers = []
        for _ in range(2 * n):
            states = [random.randint(0, 1) for _ in range(2 ** (n + 1))]
            transition_matrix = [[states[i] ^ states[j] for j in range(2 ** (n + 1))] for i in range(2 ** (n + 1))]
            layers.append((transition_matrix, random.choice([0, 1])))
        return layers

    def extract_M_bar_k(layers):
        M_bar_k = []
        s = len(layers[0][0])
        for layer, parity in layers:
            M_k = matrix_add(*[matrix_mul(layer, layer) if p == parity else identity_matrix(s) for p in [0, 1]])
            M_bar_k.append(matrix_sub(M_k, [[trace(M_k) / s] * s for _ in range(s)]))
        return M_bar_k

    def bfs_close_lie_algebra(M_bar_k):
        n = len(M_bar_k)
        generators = list(range(n))
        queue = deque(generators)
        visited = set(generators)
        while queue:
            current = queue.popleft()
            for other in range(n):
                if other not in visited:
                    commutator = matrix_mul(matrix_sub(M_bar_k[current], identity_matrix(len(M_bar_k))))
                    if frobenius_norm(commutator) < 1e-9:
                        visited.add(other)
                        queue.append(other)
        return len(visited)

    n_values = [3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
    results = []

    for s in [4, 8, 16, 32, 64, 128]:
        for n in n_values:
            layers = build_bp(n)
            M_bar_k = extract_M_bar_k(layers)
            rho = bfs_close_lie_algebra(M_bar_k)

```

```

        results.append({
            "metric_name": "rho",
            "metric_value": rho,
            "instances_tested": 1,
            "conjecture_holds": rho <= 4 * math.log2(s) + 10,
            "counterexample": ""
        })

canonical_layers = build_bp(12)
canonical_M_bar_k = extract_M_bar_k(canonical_layers)
rho_canonical = bfs_close_lie_algebra(canonical_M_bar_k)
results.append({
    "metric_name": "rho",
    "metric_value": rho_canonical,
    "instances_tested": 1,
    "conjecture_holds": rho_canonical >= n_values[-1] // 2,
    "counterexample": ""
})

return {
    "seed": seed,
    "metrics": results
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 1000) for _ in range(30)]

    all_results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        all_results.extend(result["metrics"])

    rho_values = [r["metric_value"] for r in all_results if "rho" in r["metric_name"]]
    support_fraction = sum(1 for r in all_results if r["conjecture_holds"]) / len(all_results)

    if support_fraction >= 0.9:
        RESULT = f"SUPPORTED mean={sum(rho_values) / len(rho_values):.2f} std={math.sqrt(sum((x -
    elif any(not r["conjecture_holds"] for r in all_results):
        first_failing_seed = next(s for s, r in zip(seeds, all_results) if not r["conjecture_holds"])
        RESULT = f"FALSIFIED counterexample=\"{next(r['counterexample'] for r in all_results if not
    else:
        RESULT = "INCONCLUSIVE insufficient evidence"

    print(RESULT)

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out after 240s without producing a RESULT line, so neither the support nor falsification condition could be evaluated. Per the rules, a timeout yields INCONCLUSIVE.
| next: Reduce the computational load (e.g., cap n at 8, reduce seeds to 10, and use sparse/structured commutator computations or early-exit when nilpotency class exceeds the threshold) and rerun with a longer timeout.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	166810
2	preregistration	claude_max	opus	0	0	7899
3	novelty	claude_max	opus	0	0	3156
4	novelty	claude_max	opus	0	0	8043
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23690
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20030
7	judge	claude_max	opus	0	0	5206

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 234835 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/11647c329d11.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/11647c329d11.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/11647c329d11.tar.gz` (if generated)